

Intro to DL4MSc: Attention and Transformer Blocks

Alexander Binder

December 13, 2025

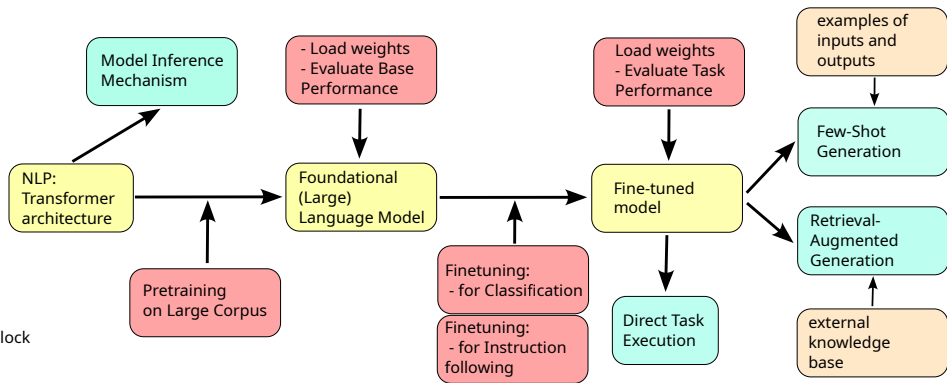
- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

Data Preparation:

- Tokenization
 - Encoding
 - Decoding
- Dataset class
 - Minibatch Sampling

Arch:

- Embedding Layer
- Attention Layer
- Decoder Transformer Block



- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

sources:

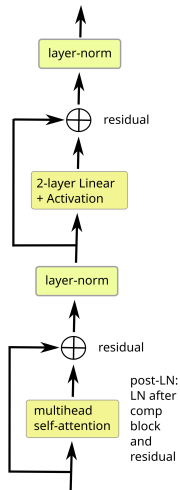
- for some code (not fast but instructive): https://github.com/rasbt/LLMs-from-scratch/blob/main/ch03/01_main-chapter-code/ch03.ipynb
- see flash-attention for fast implementations

- ⊙ we consider a setup of next word/token prediction
 - The → vanilla
 - The vanilla → cake
 - The vanilla cake → with
 - The vanilla cake with → cherries
 - The vanilla cake with cherries → tasted
 - The vanilla cake with cherries tasted → like
 - The vanilla cake with cherries tasted like → heaven
 - The vanilla cake with cherries tasted like heaven → .
 - The vanilla cake with cherries tasted like heaven. → <'ENDOFTEXT'>
- ⊙ We could design a neural network using 1D-convolutions, predict the next token using a classification head with as many classes as we have elements in the vocabulary.
- ⊙ This has been used indeed, e.g. TCNs as in Bai et al <https://arxiv.org/pdf/1803.01271>. Alternatives would be recurrent neural networks such as the LSTM (Schmidhuber et al, 1997, <https://www.bioinf.jku.at/publications/older/2604.pdf>).

- we consider a setup of next word/token prediction
 - The $\rightarrow$ vanilla
 - [omit some steps ...]
 - The vanilla cake with cherries tasted like $\rightarrow$ heaven
 - The vanilla cake with cherries tasted like heaven $\rightarrow$.
 - The vanilla cake with cherries tasted like heaven. $\rightarrow$ <'ENDOFTEXT'>
- We could design a neural network using 1D-convolutions, predict the next token using a classification head with as many classes as we have elements in the vocabulary.
- difference text vs. image
 - image: single pixel in one image often not crucial. Exist neighboring pixels which are correlated, contain similar, partially redundant information.
locally correlated information can be amplified in the next layer by convolution with a localized kernel of small kernel size ...
dropping a few pixels often not a problem.
 - text: key tokens usually not redundant (e.g. subject, main verb etc).
Dropping a key token loses information. e.g. I was doing -¿ I [] doing. (am / was ?)

- we consider a setup of next word/token prediction
 - The $\rightarrow$ vanilla
 - [omit some steps ...]
 - The vanilla cake with cherries tasted like $\rightarrow$ heaven
 - The vanilla cake with cherries tasted like heaven $\rightarrow$.
 - The vanilla cake with cherries tasted like heaven. $\rightarrow$ <'ENDOFTEXT'>
- We could design a neural network using 1D-convolutions, predict the next token using a classification head with as many classes as we have elements in the vocabulary.
- Issue with 1D-CNNs: be able to retrieve a token at the start in a long sequence
 - field of view in one layer is limited by kernel size
 - need to stack many layers for large field of view
 - kernel = fixed weighting of features
 - a single important token not guaranteed be transported through all the layers , can get lost in a fixed-weight summing

We will build a network as a sequence of so-called **transformer decoder blocks**.



- attention layer:
 - computes a feature as a linear combination of features from all preceding tokens inputted into the network ... using data-dependent weights

- we consider a setup of next word/token prediction
 - The $\rightarrow$ vanilla
 - [omit some steps ...]
 - The vanilla cake with cherries tasted like $\rightarrow$ heaven
 - The vanilla cake with cherries tasted like heaven $\rightarrow$.
 - The vanilla cake with cherries tasted like heaven. $\rightarrow$ <'ENDOFTEXT'>
- attention: data-dependent weighting of features ... can learn to attend to what is needed

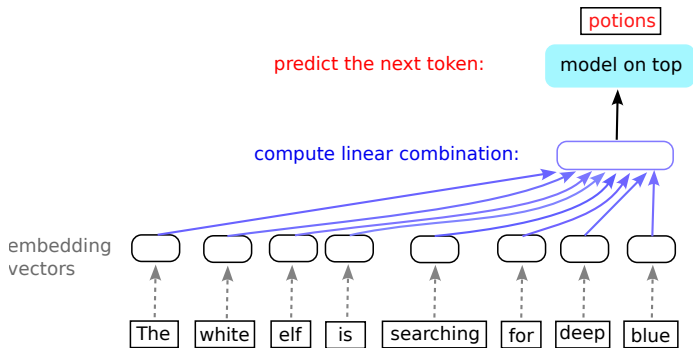
[bla bla bla ...] Alex has a friend called John. He is 46 years of age. Alex has a best friend named Mary. She is 48 years young. Alex has a bunch of orange cats who bully him as his enemies. They are 2 or 3 years old. [... bla bla bla]

How old is the best friend of Alex ? [Generate an answer here] The age of his best friend is [...]

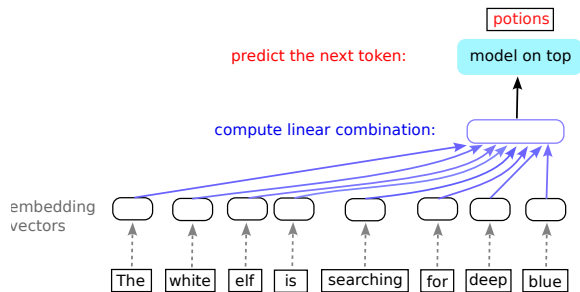
- attention: data-dependent weighting of features ... can learn to attend to what is needed depending on the last words.

- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

- ⊙ attention: data-dependent weighting of features ... able to learn to attend to the key token, e.g. the single verb
- ⊙ three types here:
 - vanilla (single-head, non-causal) attention
 - multi-head attention
 - causal attention (important for sequence generation!!)

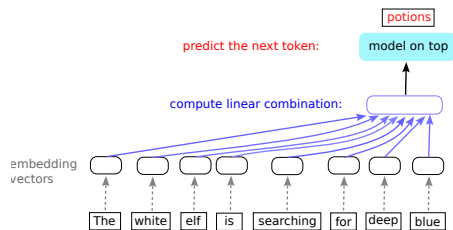


- suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- We consider a layer in the network, where we have features $x_0, \dots, x_{s-1}$ for the already generated tokens (e.g. 1st



- suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- We have a layer with features $x_0, \dots, x_{s-1}$ for the already generated tokens
- try 1: a data-dependent weighting attending to the feature of the last generated token would be

$$a(s-1) = \sum_{l=0}^{s-1} w_l(x_{s-1})x_l$$

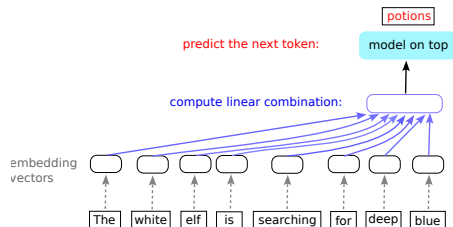


- suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- We have a layer with features $x_0, \dots, x_{s-1}$ for the already generated tokens
- try 2: the weight w_l for the l -th feature x_l should depend on the similarity between x_l and x_{s-1}

$$a(s-1) = \sum_{l=0}^{s-1} w_l(x_{s-1}) x_l$$

$$w_l(x_{s-1}) = \text{sim}(x_l, x_{s-1})$$

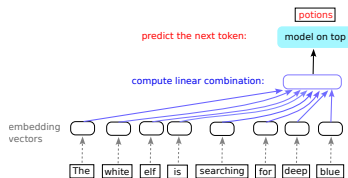
$$\Rightarrow a = \sum_{l=0}^{s-1} \text{sim}(x_l, x_{s-1}) x_l$$



- suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- We have a layer with features $x_0, \dots, x_{s-1}$ for the already generated tokens
- try 2: the weight w_l for the l -th feature l_k should depend on the similarity between x_l and x_{s-1}

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(x_l, x_{s-1}) x_l$$

- we want the similarity to be able to depend on only a part of the features x_l, x_{s-1}



- suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- We have a layer with features $x_0, \dots, x_{s-1}$ for the already generated tokens
- try 3: we want the similarity between x_l and x_{s-1} to be able to depend on only a part of the features x_l, x_{s-1}

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(x_l, x_{s-1}) x_l$$

- replace x_l in the similarity term by a linear projection $F^{(k)}(x_l) = W^{(k)}x_l + b^{(k)}$

- try 3: we want the similarity between x_l and x_{s-1} to be able to depend on only a part of the features x_l, x_{s-1}

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(x_l, x_{s-1}) x_l$$

- replace x_l in the similarity term by a linear projection $F^{(k)}(x_l) = W^{(k)}x_l + b^{(k)}$
- replace x_{s-1} in the similarity term by another linear projection $F^{(q)}(x_{s-1}) = W^{(q)}x_{s-1} + b^{(q)}$

$$\Rightarrow a(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1})) x_l$$

- linear projections allow to filter out a part of the features x_l and x_{s-1} . Note: they are trainable via $W^{(k)}, W^{(q)}$!

- we want the similarity between x_l and x_{s-1} to be able to depend on only a part of the features x_l, x_{s-1}

$$a = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1}))x_l$$

- try 4: additionally we want the final accumulated feature to depend on only a part of the features x_l
- replace x_l in the weighted feature term also by $F^{(v)}(x_l) = W^{(v)}x_l + b^{(v)}$

$$\Rightarrow a(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1}))F^{(v)}(x_l)$$

- ⦿ suppose we have generated s tokens $z_0, \dots, z_{s-1}$. We want to generate the next token z_s .
- ⦿ We consider a layer in the network, where we have features $x_0, \dots, x_{s-1}$ for the already generated tokens (e.g. 1st layer it would be the word embeddings)
- ⦿ we obtain a model for a single attention head, which uses three trainable linear projections:

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1})) F^{(v)}(x_l)$$

$$F^{(k)}(x_l) = W^{(k)}x_l + b^{(k)}$$

$$F^{(q)}(x_{s-1}) = W^{(q)}x_{s-1} + b^{(q)}$$

$$F^{(v)}(x_l) = W^{(v)}x_l + b^{(v)}$$

Intuition for attention for sequence generation:

Learn a weighted sum of features which attends to the last generated token. It should capture relevant context from the preceding tokens – by the weighted summing of their features. To attend means: The context should depend on the last generated token.

- we obtain a model for a single attention head, which uses three trainable linear projections:

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1})) F^{(v)}(x_l)$$

$$F^{(k)}(x_l) = W^{(k)}x_l + b^{(k)}$$

$$F^{(q)}(x_{s-1}) = W^{(q)}x_{s-1} + b^{(q)}$$

$$F^{(v)}(x_l) = W^{(v)}x_l + b^{(v)}$$

We can generalize it using the query-key-value model of attention:

- query q - a feature on which the weights should depend
- key-value pairs (k, v) :
 - values v are the actual content to be accumulated ()
 - keys k are the identifiers for the values (e.g. a name)
- we compute a similarity between the query (token/feature) and each of the keys and use that to make weights for the values .

$$a(q) = \sum_{l: (k_l: v_l)} \text{sim}(q, k_l) v_l$$

- we obtain a model for a single attention head, which uses three trainable linear projections:

$$a(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k)}(x_l), F^{(q)}(x_{s-1})) F^{(v)}(x_l)$$

$$F^{(k)}(x_l) = W^{(k)}x_l + b^{(k)}$$

$$F^{(q)}(x_{s-1}) = W^{(q)}x_{s-1} + b^{(q)}$$

$$F^{(v)}(x_l) = W^{(v)}x_l + b^{(v)}$$

This can be mapped onto the general formulation

$$a(q) = \sum_{l: (k_l, v_l)} \text{sim}(q, k_l) v_l$$

using:

$$q = F^{(q)}(x_{s-1})$$

$$k_l = F^{(k)}(x_l)$$

$$v_l = F^{(v)}(x_l)$$

$$q = F^{(q)}(x_{s-1})$$

$$k_l = F^{(k)}(x_l)$$

$$v_l = F^{(v)}(x_l)$$

Self-attention:

When queries, keys and values are taken from the same feature block

next step: example for the similarity

$$a(q) = \sum_{l: (k_l, v_l)} \text{sim}(q, k_l) v_l$$

- how can the similarity look like ? For example

$$\begin{aligned} \text{sim}(q, k_l) &= \text{softmax}_{l'=0, \dots, s-1} \left(\underbrace{F^{(q)}(q) \cdot F^{(k)}(x_{l'})}_{\text{inner prod.}} D^{-1/2} \right) [l] \\ &= \text{softmax}_{l'=0, \dots, s-1} \left(\underbrace{F^{(q)}(q)^\top F^{(k)}(x_{l'})}_{\text{matmul}} D^{-1/2} \right) [l] \end{aligned}$$

- A softmax of inner products between the transformed query feature $F^{(q)}(q)$ and the key feature $F^{(k)}(x_{l'})$ for all possible sequence elements $x_{l'}$
- scaling with $D^{-1/2}$ empirically much better. D is the dimension of the transformed vectors.

General formulation for any set of key value pairs $\{(k_l, v_l)\}$ beyond sequences with a temporal/causal order:

$$a(q) = \sum_{l: (k_l, v_l)} \text{sim}(q, k_l) v_l$$

$q = F^{(q)}(x^{(q)})$ query base feature

$k_l = F^{(k)}(x_l^{(k)})$, $x_l^{(k)}$ l-th key base feature

$v_l = F^{(v)}(x_l^{(v)})$, $x_l^{(v)}$ l-th value base feature

Intuition for qkv-attention in general:

Learn a weighted sum of features which attends to the query token. It should capture relevant context from the value features. The weight for a value feature v_l depends on the similarity between its associated key k_l and the query k . Typically query-, key- and value-features are obtained from affine mappings of query-, key- and value base features (denoted above as $x^{(q)}, x_l^{(k)}, x_l^{(v)}$), in order to make the similarity trainable.

Why qkv-attention as model ?

- ⊙ Allows to model more general interactions beyond the example of attending to the last generated token in a sequence. For example:
 - query = text token
 - keys and values are taken from an image.
 - Can be used to answer questions posed to an image. Need to grab image content suitable to a keyword in a question.

See Figure 3 in Rahman et al, CVPR WS 2021

https://openaccess.thecvf.com/content/CVPR2021W/MULA/papers/Rahman_An_Improved_Attention_for_Visual_Question_Answering_CVPRW_2021_paper.pdf

Cross-attention:

definition further below :)

An example for cross-attention between text modalities: translation: EN $\rightarrow$ GER

Ask yourself:

- query feature from the source or target language?
- key-value features from the source or target language?

- ◉ Here an example for cross-attention between image features for different image scales:
https://openaccess.thecvf.com/content/ICCV2021/papers/Chen_CrossViT_Cross-Attention_Multi-Scale_Vision_Transformer_for_Image_Classification_ICCV_2021_paper.pdf
- ◉ Cross-attention in diffusion models:
 - an early paper but not easily readable: page 5 of Gu et al. <https://arxiv.org/pdf/2111.14822>
 - more clear here: Figure 2 and page 3 describing it for StableDiffusion Liu et al. https://openaccess.thecvf.com/content/CVPR2024/papers/Liu_Towards_Understanding_Cross_and_Self-Attention_in_Stable-Diffusion_for_Text-Guided_CVPR_2024_paper.pdf

self vs cross-attention

- self-attention: When queries and keys-value-pairs are taken from the same feature block
- cross-attention: When queries come from a different feature block or source than the key-value pairs

In general:

- keys and values are semantically connected.
- queries can come from the same modality/source as k, v or from a different one
- if taken from the same source, keys k and values v of a pair (k_l, v_l) *are not the exact same feature*, but obtained by different functions from the source (possibly from the same base feature). reason: they serve different tasks:
 - value should contain information for the target prediction task
 - key ... should contain information for matching the query features
 - using the same mapping for keys and values is not guaranteed to be a good idea, too

next: non-causal self-attention in code.

We compute the attentions for a sequence of queries derived from base feature

`x.shape=(batch_size, seqlen, dim_in)`

- This is we have a base feature which is a sequence: $x[:, 0, :], \dots, x[:, seqlen - 1, :]$ corresponding to the first $seqlen$ generated words
- when applying $F^{(q)}$, we obtain a sequence of query features $Q[:, 0, :], \dots, Q[:, seqlen - 1, :]$
- when applying $F^{(k)}$, we obtain a sequence of key features $K[:, 0, :], \dots, K[:, seqlen - 1, :]$

similarity matrix for a single element in the batch
its shape is: (seqlen,seqlen)

	k_0	k_1	k_2	$\dots$	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
$\vdots$					
$\vdots$					
$\vdots$					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

next: non-causal self-attention in code.

We compute the attentions for a sequence of queries derived from base feature

`x.shape=(batch_size, seqlen, dim_in)`

- when applying $F^{(q)}$, we obtain a sequence of query features $Q = F^{(q)}(x)$,
 $Q.shape = (bs, seqlen, dim_out)$
- when applying $F^{(k)}$, we obtain a sequence of key features $K = F^{(k)}(x)$,
 $K.shape = (bs, seqlen, dim_out)$
- the similarity matrix for one element in the batch will be of shape $(seqlen, seqlen)$

similarity matrix for a single element in the batch
its shape is: $(seqlen, seqlen)$

	k_0	k_1	k_2	$\cdots$	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
$\vdots$					
$\vdots$					
$\vdots$					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

We compute the attentions for a sequence of queries derived from base feature

$x.shape = (batch_size, seqlen, dim_in)$

- when applying $F^{(q)}$, we obtain a sequence of query features $Q = F^{(q)}(x)$,
 $Q.shape = (bs, seqlen, dim_out)$
- when applying $F^{(k)}$, we obtain a sequence of key features $K = F^{(k)}(x)$,
 $K.shape = (bs, seqlen, dim_out)$
- the similarity matrix for one element in the batch will be of shape $(seqlen, seqlen)$

similarity matrix for a single element in the batch
its shape is: $(seqlen, seqlen)$

	k_0	k_1	k_2	...	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
...					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

do the computation for one element in the batch:

- $Q.shape = (1, seqlen, dim_out)$
- $K.shape = (1, seqlen, dim_out)$
- how to multiply to obtain shape $(1, seqlen, seqlen, 1)$?

$$PreSim = QK^T = F^{(q)}(x)(F^{(k)}(x))^T$$

similarity matrix for a single element in the batch
its shape is: (seqlen, seqlen)

	k_0	k_1	k_2	...	$k_{\text{seqlen}-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
...					
$q_{\text{seqlen}-1}$	0.08	0.10	0.06		0.22

do the computation for one element in the batch:

- $Q.shape = (1, \text{seqlen}, \text{dim_out})$
- $K.shape = (1, \text{seqlen}, \text{dim_out})$
- similarity matrix:

$$\begin{aligned} Sim &= \text{softmax}(QK^\top d^{-1/2}) \\ &= \text{softmax}(F^{(q)}(x)(F^{(k)}(x))^\top d^{-1/2}) \end{aligned}$$

$$Sim.shape = (1, \text{seqlen}, \text{seqlen}, 1)$$

- $Sim.shape = (1, \text{seqlen}, \text{seqlen}, 1)$

1st seqen-axis $\rightsquigarrow$ queries,

2nd seqen-axis $\rightsquigarrow$ keys

$$Sim[0, i, l, 0] = Sim(q^{(i)}, k^{(l)})$$

- Need to compute:

$$\sum_l sim(q_i, k_l) v_l$$

- keys are aligned to values.

$$Sim.shape = (1, \text{seqlen}, \underbrace{\text{seqlen}}_{\text{axis of keys}}, 1)$$

$$V = F^{(l)}(x)$$

$$V.shape = (1, \text{seqlen}, \text{dim})$$

seqlen axis of values corresp. to keys

- so:

$$A = Sim[0, :, :, 0] V[0, :, :]$$

similarity matrix for a single element in the batch
its shape is: (seqlen, seqlen)

	k_0	k_1	k_2	...	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
...					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

do the computation for one element in the batch:

- $Q.shape = (1, seqlen, dim_out)$
- $K.shape = (1, seqlen, dim_out)$
- similarity matrix:

$$Sim = \text{softmax}(QK^T d^{-1/2})$$

$$Sim.shape = (1, seqlen, seqlen, 1)$$

- keys are aligned to values.

$$Sim.shape = (1, seqlen, \underbrace{seqlen}_{\text{axis of keys}}, 1)$$

$$V = F^{(l)}(x)$$

$$V.shape = (1, seqlen, dim)$$

$seqlen$ axis of values corresp. to keys

- so:

$$A = Sim[0, :, :, 0] V[0, :, :]$$

- in math: (cf. Vaswani et al.)

$$Att = \text{softmax}(QK^T d^{-1/2}) V$$

next: non-causal self-attention in code.

We compute the attentions for a sequence of queries derived from base feature

`x.shape=(batch_size, seqlen, dim_in)`

- when applying $F^{(q)}$, we obtain a sequence of query features $Q = F^{(q)}(x)$,
 $Q.shape = (bs, seqlen, dim_out)$
- when applying $F^{(k)}$, we obtain a sequence of key features $K = F^{(k)}(x)$,
 $K.shape = (bs, seqlen, dim_out)$
- we need a **similarity matrix for each batch element**, thus $sim.shape = (bs, seqlen, seqlen)$

similarity matrix for a single element in the batch
its shape is: (seqlen,seqlen)

	k_0	k_1	k_2	...	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
...					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

next: non-causal self-attention in code.

We compute the attentions for a sequence of queries derived from base feature

$x.shape = (batch_size, seqlen, dim_in)$

- we obtain for query features
 $Q.shape = (bs, seqlen, dim_out)$
- we obtain for key features
 $K.shape = (bs, seqlen, dim_out)$
- we need a similarity matrix for each batch element, thus $sim.shape = (bs, seqlen, seqlen)$

$(bs, seqlen, dim_out) \star (bs, seqlen, dim_out)$

$\xrightarrow{magic?} (bs, seqlen, seqlen)$??How-to

similarity matrix for a single element in the batch

its shape is: $(seqlen, seqlen)$

	k_0	k_1	k_2	$\cdots$	$k_{seqlen-1}$
q_0	0.15	0.07	0.05		0.11
q_1	0.03	0.18	0.07		0.08
q_2	0.06	0.03	0.13		0.09
$\vdots$					
$q_{seqlen-1}$	0.08	0.10	0.06		0.22

- good news: `torch.matmul` does batched matrix multiplication, see:
<https://docs.pytorch.org/docs/stable/generated/torch.matmul.html>

non-causal single head attention

```
class SelfAttention_singlehead(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias) #check nn.Linear doc!
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)  #(bs, seqlen, d_out)
        queries = self.W_query(x)  #(bs, seqlen, d_out)
        values = self.W_value(x)  #(bs, seqlen, d_out)
        attn_scores = queries @ keys.T  #does batched matmul! (bs, seqlen, seqlen)
         # https://docs.pytorch.org/docs/stable/generated/torch.matmul.html
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1
        )  #(bs, seqlen, seqlen)
        context_vec = attn_weights @ values  #check here shape, again batched matmul
        return context_vec
```

next: multi-head attention as simple extension of vanilla attention

"Alex has a friend John who is vegetarian and a friend Mary who loves huge bloody steaks. Alex who likes to troll his friends cooked a huge piece of tofu that tasted like toilet paper for "

[next token here:] **BOTH**

- insight: sometimes to generate the next word one needs to capture more than one context.
- Multihead attention: compute attention R times for the *same query base feature* $x^{(q)}$ but *with R different (trainable) weights*. Each of the R outputs is called one attention head.
- take the same query base feature, also the same key-and value base features, compute R times with different trainable weights.

$$a^{(r)}(s-1) = \sum_{l=0}^{s-1} \text{sim}(F^{(k,r)}(x_l), F^{(q,r)}(x_{s-1})) F^{(v,r)}(x_l), \quad \text{for } r = 0, \dots, R-1$$

$$F^{(k,r)}(x_l) = W^{(k,r)} x_l + b^{(k,r)}$$

$$F^{(q,r)}(x_{s-1}) = W^{(q,r)} x_{s-1} + b^{(q,r)}$$

$$F^{(v,r)}(x_l) = W^{(v,r)} x_l + b^{(v,r)}$$

simplified non-causal multihead attention

```
class SelfAttention_multihead_aswrapper(nn.Module):  
  
    def __init__(self, d_in, d_out, num_heads, qkv_bias=False):  
        super().__init__()  
        self.heads = nn.ModuleList(  
            [SelfAttention_singlehead(d_in, d_out, qkv_bias) for _ in range(num_heads)]  
        )  
  
    def forward(self, x):  
        return torch.cat([head(x) for head in self.heads], dim=-1)
```

more efficient by using a large weight vector and splitting its outputs into multiple heads

```
class SelfAttention_multihead_v2(nn.Module):
    def __init__(self, d_in, d_out, num_heads, dropout, qkv_bias=False):
        super().__init__()
        assert (d_out % num_heads == 0), "d_out must be divisible by num_heads"
        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias) #check nn.Linear doc!
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # code see below
        return context_vec
```

```

class SelfAttention_multihead_v2(nn.Module):
    def __init__(self, d_in, d_out, num_heads, qkv_bias=False):
        pass # code see above
    def forward(self, x):
        b, num_tokens, d_in = x.shape
        keys = self.W_key(x)      #(bs, seqlen, d_out)
        queries = self.W_query(x) #(bs, seqlen, d_out)
        values = self.W_value(x)  #(bs, seqlen, d_out)

        #add axis for heads, split into separate heads along this axis:
        keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)      #(bs, seqlen, n_heads, head_dim)
        values = values.view(b, num_tokens, self.num_heads, self.head_dim) # should be chunks of self.head_dim size, so this one is last
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)

        keys = keys.transpose(1, 2)      #(bs, n_heads, seqlen, head_dim)
        queries = queries.transpose(1, 2) #(bs, n_heads, seqlen, head_dim)
        values = values.transpose(1, 2)  #(bs, n_heads, seqlen, head_dim)

        #(bs, n_heads, seqlen, head_dim) * (bs, n_heads, head_dim, seqlen)
        attn_scores = queries @ keys.transpose(2, 3) # batched, so (bs, n_heads, seqlen, seqlen)

        attn_weights = torch.softmax( attn_scores / keys.shape[-1]**0.5, dim=-1)
        attn_weights = self.dropout(attn_weights)

        #(bs, n_heads, seqlen, seqlen) * #(bs, n_heads, seqlen, head_dim)
        context_vec = attn_weights @ values      #(bs, n_heads, seqlen, head_dim)
        context_vec = context_vec.transpose(1, 2) #(bs, seqlen, n_heads, head_dim)
        context_vec = context_vec.contiguous().view(b, num_tokens, self.d_out)  #(bs, seqlen, n_heads*head_dim)
        return context_vec

```

next: causal attention

- the main reason to use causal attention is efficient inference when doing sequence generation

- the main reason to use causal attention is efficient inference when doing sequence generation

Our setup - generating a sentence, e.g. as answer, fits into sequence generation use cases

- causal means here: a query q_i has 0 similarity to all keys k_l in the future ($i < l$)

$$\text{sim}(q_i, k_l) = 0 \text{ if } i < l$$

- similarity matrix will have all zeros on the upper diagonal
- a query can be matched only to keys from 0 until the time-step of the query

similarity matrix for a single element in the batch
its shape is: (seqlen,seqlen)

$$\text{causal: } \text{sim}(q_i, k_l) = 0 \text{ if } i < l$$

	k_0	k_1	k_2	$\cdots$	$k_{\text{seqlen}-1}$
q_0	0.15	0.00	0.00		0.00
q_1	0.03	0.18	0.00		0.00
q_2	0.06	0.03	0.13		0.00
$\vdots$					
$q_{\text{seqlen}-1}$	0.08	0.10	0.06		0.22

- the main reason to use causal attention is efficient inference when doing sequence generation

similarity matrix for a single element in the batch
its shape is: (seqlen,seqlen)

causal: $\text{sim}(q_i, k_l) = 0$ if $i < l$

- causal means here: a query q_i has 0 similarity to all keys k_l in the future ($i < l$)

$$\text{sim}(q_i, k_l) = 0 \text{ if } i < l$$

- implementation for softmax: set attention scores to $-\infty$ before applying softmax (setting to zero after softmax will require another compute step! Think why!)

	k_0	k_1	k_2	$\cdots$	$k_{\text{seqlen}-1}$
q_0	0.15	0.00	0.00		0.00
q_1	0.03	0.18	0.00		0.00
q_2	0.06	0.03	0.13		0.00
$\vdots$					
$q_{\text{seqlen}-1}$	0.08	0.10	0.06		0.22

```
class CausalAttention(nn.Module):
    def __init__(self, d_in, d_out, context_length, dropout, qkv_bias=False):
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            'mask', torch.triu(torch.ones(context_length, context_length), diagonal=1) #1 on upper diagonal
        ) #returned with weights, can be saved but not trainable

    def forward(self, x):
        b, num_tokens, d_in = x.shape

        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)

        attn_scores = queries @ keys.transpose(1, 2)

        attn_scores.masked_fill_(
            self.mask.bool()[:num_tokens, :num_tokens], -torch.inf) # the new addition!

        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1)
        attn_weights = self.dropout(attn_weights)
        context_vec = attn_weights @ values

        return context_vec
```


- the main reason to use causal attention is efficient inference when doing sequence generation
- If attention weights A are causal, then the weights until token k , $A[:, k-1, : k-1]$, will never change, no matter what tokens will be added after k : $x_k, x_{k+1}, x_{k+2} \dots$. Simply because weights for similarities to those future tokens are $= 0$.

similarity matrix for a single element in the batch
its shape is: (seqlen,seqlen)

causal: $\text{sim}(q_i, k_l) = 0$ if $i < l$

	k_0	k_1	k_2	$\longrightarrow$	$k_{\text{seqlen}-1}$
q_0	0.15	0.00	0.00		0.00
q_1	0.03	0.18	0.00		0.00
q_2	0.06	0.03	0.13		0.00
$\vdots$					
$\vdots$					
$\vdots$					
$q_{\text{seqlen}-1}$	0.08	0.10	0.06		0.22

- Therefore the weighted sum of value features until token k will never change, no matter what tokens will be added after k : $x_k, x_{k+1}, x_{k+2} \dots$

$$a(k) = \sum_{l=0}^{k-1} \text{sim}(q_k, k_l) v_l + \sum_{l=k}^T 0 v_l$$

- now assume: all other steps/layers in the neural network do not mix in future information
- this means one can reuse the resulting weighted features $a(l), l \leq k$ until token k for all future tokens $x_k, x_{k+1}, x_{k+2} \dots$ without recomputing the weighted features until k ! This means one can cache those weighted features for predicting on future tokens $x_k, x_{k+1}, x_{k+2} \dots$

When using causal attention (see above additional condition), one needs to compute the weighted features for step k only once, and can reuse them for future tokens from a cache. This has an implication what really needs to be computed in each step.

see a later section for more

No, when sequence generation is not intended - e.g. in toxicity prediction, source language encoding in translation.

Encoder-models like BERT do not use causal attention. BERT Devlin et al.

<https://arxiv.org/abs/1810.04805> uses vanilla attention and thus sees the whole sequence. More in the next lecture.

- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

Ba et al. <https://arxiv.org/abs/1607.06450>

- ⊙ Normalizes a feature map, by subtracting a mean computed over parts of a feature map, by dividing a standard deviation, computed over the same parts, then applies an affine transform

$$\tilde{x} = \frac{x - E[x]}{\sqrt{\text{Var}(x) + \epsilon}} * \gamma + \beta$$

Difference to Batchnorm:

- ⊙ Layernorm: mean and variance used to subtract from a feature are computed for the current sample and for the current token separately (preserves causality, usable at testing time), and pooled over all features in one layer belonging to one token.
 - <https://docs.pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html>.
One could apply layernorm to pool over tokens, but this is not done in sequential tasks.
- ⊙ Batchnorm: computes mean and variance pooled over a mini-batch of samples, and for each feature channel separately, but maybe also pooled over all tokens (breaks causality).

Ba et al. <https://arxiv.org/abs/1607.06450>

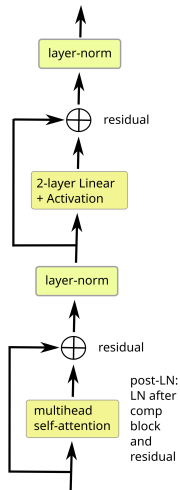
- ⊙ Normalizes a feature map, by subtracting a mean computed over parts of a feature map, by dividing a standard deviation, computed over the same parts, then applies an affine transform

$$\tilde{x} = \frac{x - E[x]}{\sqrt{\text{Var}(x) + \epsilon}} * \gamma + \beta$$

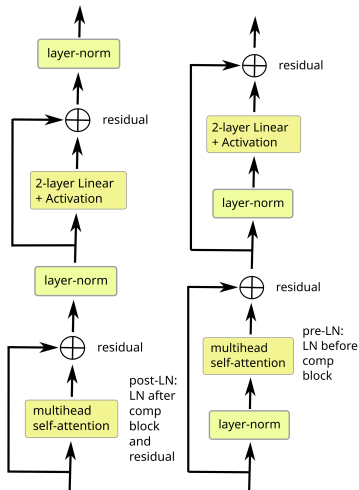
- A nice graphic can be found here: <https://stats.stackexchange.com/questions/474440/why-do-transformers-use-layer-norm-instead-of-batch-norm>
- A paper analyzing issues with batchnorm in NLP https://proceedings.neurips.cc/paper_files/paper/2022/file/f4f2f2b3c67da711df6df557fc870c4a-Paper-Conference.pdf however its findings are not fully conclusive.

- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

We will build a network as a sequence of so-called **transformer decoder blocks**.

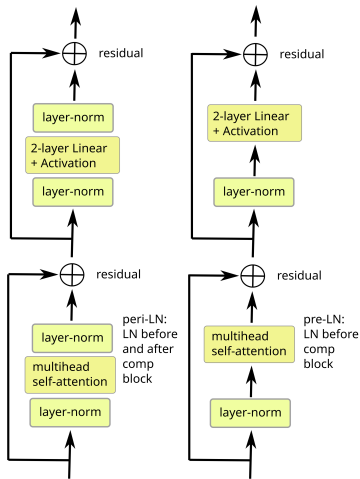


- important: MLP is applied for each token separately
- why MLP? MLP for $feature.shape = (bsize, seqlen, dim)$ applies the simplest non-linearity onto the attention features
- the attention has mixed content across tokens and captured context from other tokens
- layernorm can be applied before the attention/mlp blocks (pre-LN) or after (post-LN) or sandwiching each block (peri-LN)



<https://arxiv.org/pdf/2002.04745>

- post-LN (left) requires a learning rate warmup for training
- note: LN in post-LN used after the residual sum

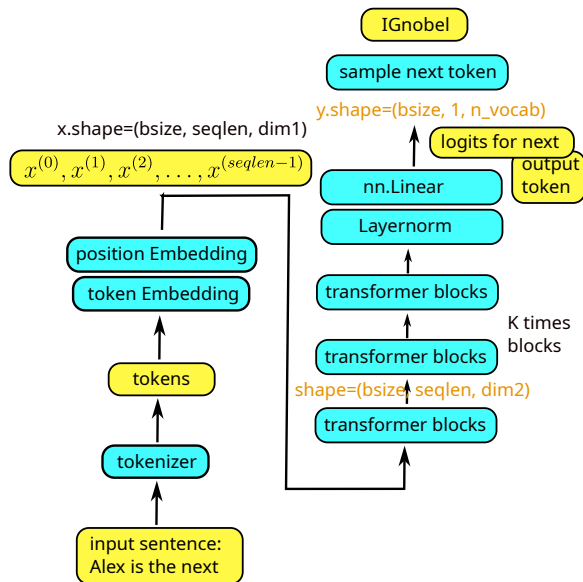


Peri-LN (left)

<https://arxiv.org/html/2502.02732v1>

- 2x LN: before and right after each Attention and MLP block

- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model**
- ⑦ Flash-Attention



Decoder for sequential generation

- when got the logits, sample the next token
 - for simplicity for now: pretend to use argmax (not done in practice!)
- iterate this step until one samples an ' $\langle \text{EOS} \rangle$ ' token. If that is sampled, stop generation.
 - during training the model will learn to associate the ' $\langle \text{EOS} \rangle$ ' token with end of generation.

- ① The high level plan
- ② Why attention?
- ③ Attention layers
- ④ Layernorm
- ⑤ One transformer block
- ⑥ The whole decoder transformer model
- ⑦ Flash-Attention

Dao et al. <https://arxiv.org/pdf/2205.14135> See Section 4 there for results

Dao et al. <https://arxiv.org/abs/2307.08691>

The exact details of Flash-Attention are not Exam stuff. What you need to know for exams is:

Flash-Attention

- speed up of attention layer computation by employing GPU-hardware specific details: onGPU-SRAM is very small but much faster than GPU-RAM
- do not instantiate the full attention weight matrix in GPU-RAM
- double looping over blocks of (k, v) and over query blocks, compute blocks of attention-weighted values on SRAM, then copy over to GPU-RAM